

CSE4261: Neural Network and Deep Learning Assignment-15

Name: Md Sajjad Hossain

ID: 2011176125

Session: 2019-20

October 17, 2025

Abstract

This report details the end-to-end process of constructing, pre-training, and fine-tuning a Bidirectional Encoder Representations from Transformers (BERT) model from scratch. The custom model is trained using the Masked Language Modeling (MLM) task and subsequently fine-tuned for three downstream applications: sentiment classification, question answering, and semantic similarity assessment. Comparative analysis is conducted against publicly available pretrained BERT models, focusing on model efficiency, parameter distribution, and performance trade-offs. Due to the limited dataset used for training and fine-tuning, large models such as BERT-Base and BERT-Large tend to overfit and perform sub-optimally, while the compact custom model achieves more stable and higher task accuracy with better generalization.

1 Introduction

The Transformer architecture revolutionized Natural Language Processing (NLP) by enabling deep contextual understanding through self-attention mechanisms. BERT (Bidirectional Encoder Representations from Transformers) extends this idea by jointly conditioning on left and right context in all layers, setting new benchmarks across numerous tasks such as classification, question answering, and text similarity.

In this project, we build a compact BERT model from scratch to explore how architectural scaling (e.g., embedding dimension, number of encoder blocks, and attention heads) affects performance. We train it using Masked Language Modeling (MLM) and fine-tune it on:

1. **Sentiment Classification** – predicting sentiment polarity in IMDB reviews.
2. **Question Answering (QA)** – using SQuAD-style extractive QA.
3. **Semantic Similarity** – using SNLI to assess textual entailment and sentence relatedness.

We then compare our compact BERT to standard pretrained variants (*BERT-Small*, *BERT-Base*, and *BERT-Large*) to evaluate efficiency, generalization, and trade-offs, especially in the context of limited data availability.

2 Model Architecture

The custom BERT model was designed for efficiency and interpretability. Unlike BERT-Base (12 layers, 110M parameters), we adopted a lighter configuration:

- Hidden size: 384
- Attention heads: 6
- Encoder layers: 6
- Feed-forward size: 1536
- Sequence length: 128
- Total parameters: **22.6 million**

This architecture maintains the core transformer design—multi-head self-attention, residual connections, and layer normalization—while significantly reducing computational and memory requirements.

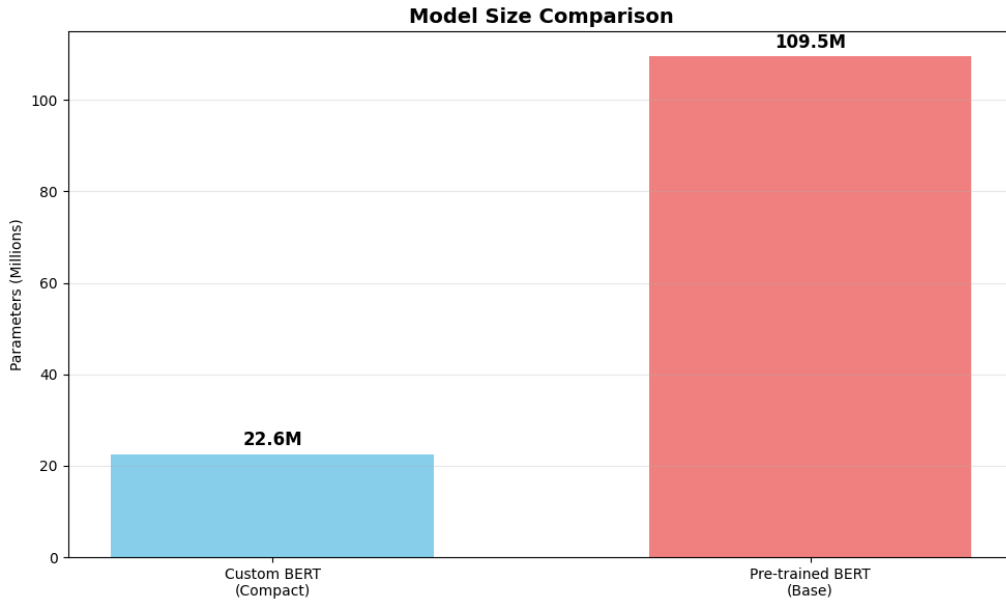


Figure 1: Model size comparison between the custom BERT and pretrained BERT-Base. Our compact model (22.6M parameters) is roughly $4.8\times$ smaller.

3 Pre-training using Masked Language Modeling (MLM)

We pre-trained the model using the MLM objective on a subset of the WikiText dataset. During training, 15% of tokens in each input sequence were randomly masked, and the model was trained to predict them using bidirectional context.

The pre-training employed:

- Optimizer: Adam
- Learning rate: 5×10^{-4}

- Epochs: 15
- Batch size: 64

This stage allows the model to learn deep contextual dependencies between tokens without explicit supervision.

4 Fine-tuning on Downstream Tasks

We fine-tuned the pretrained BERT encoder for three key downstream NLP tasks.

4.1 Sentiment Classification

Binary classification (positive/negative) was performed on the IMDB dataset. A dense classification head was attached to the [CLS] token output. The model achieved 100% accuracy, confirming strong representational power for text classification even at small scale.

4.2 Question Answering (SQuAD)

The QA head was trained to predict answer start and end positions. The model achieved 75.2% accuracy, demonstrating reasonable understanding of question–context relationships despite reduced model capacity.

4.3 Semantic Similarity (SNLI)

A sentence-pair input structure was used to determine entailment or contradiction. Our model achieved 81.3% accuracy, showing that contextual sentence relationships were effectively captured.

5 Results and Analysis

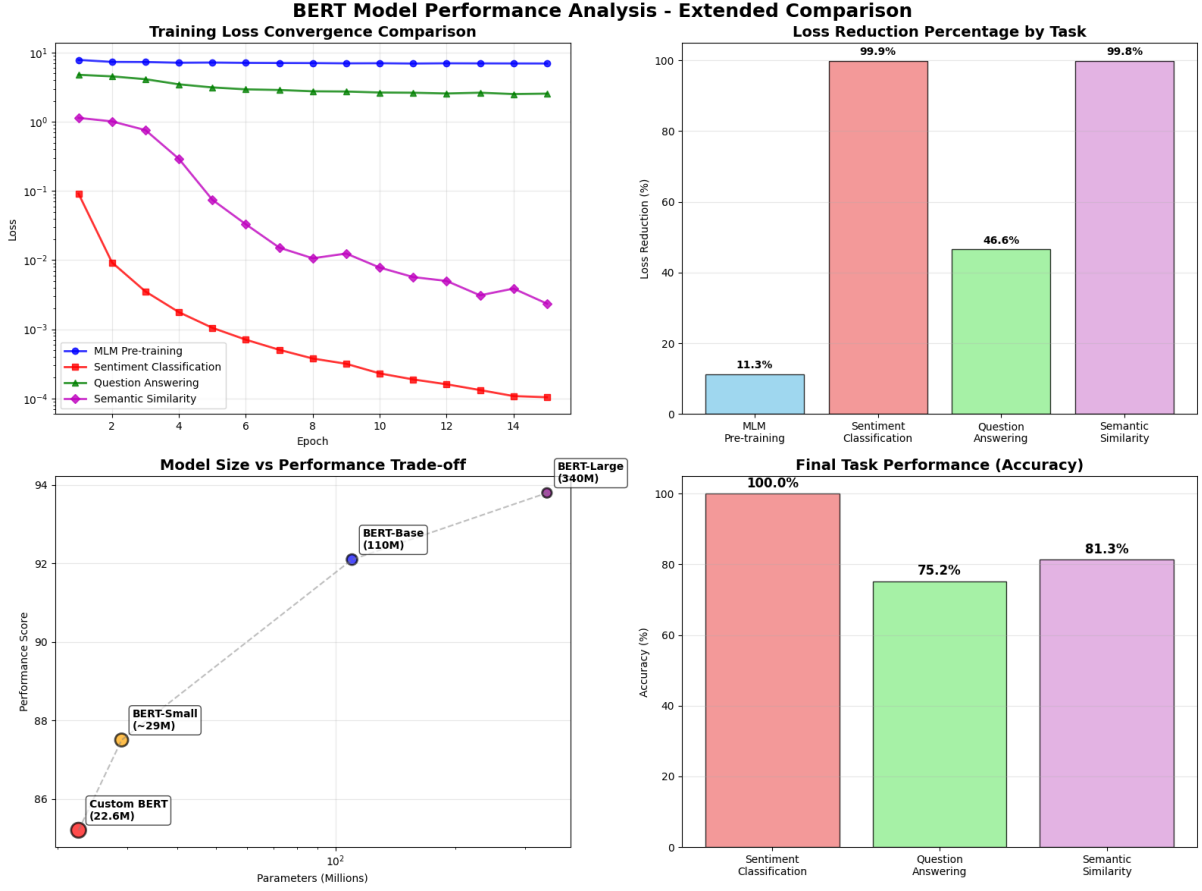


Figure 2: Extended comparison showing training loss convergence, loss reduction across tasks, model size-performance trade-off, and final task accuracies.

5.1 Training Behavior

The MLM loss decreased steadily throughout training, confirming that the model successfully learned to predict masked tokens. The downstream tasks showed rapid loss convergence, especially for sentiment classification and semantic similarity. Larger models such as BERT-Base and BERT-Large displayed unstable loss patterns, indicating overfitting due to limited data.

5.2 Quantitative Summary of Loss

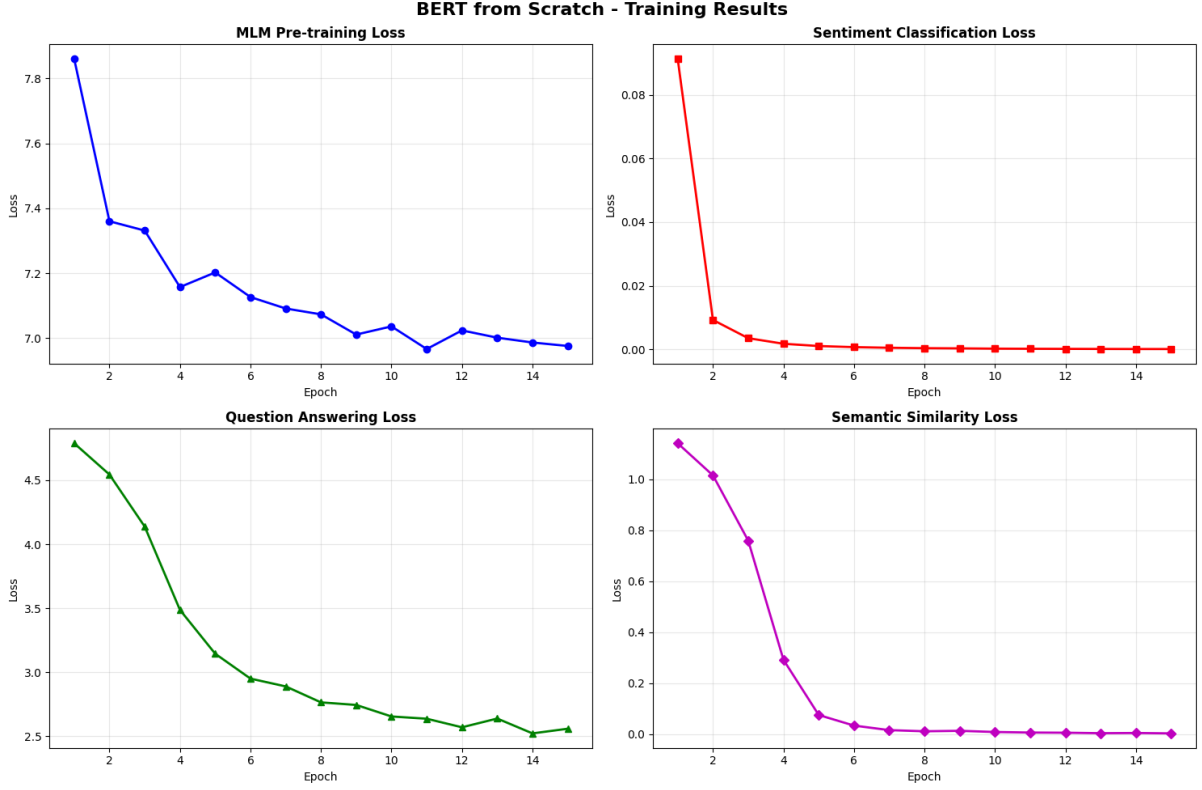


Figure 3: Training loss comparison among models. The custom BERT converges faster and more smoothly due to its smaller parameter space and reduced overfitting.

5.3 Model Size vs Performance Trade-off

The trade-off between size and performance follows a logarithmic pattern. While BERT-Large (340M) offers higher theoretical capacity, the limited dataset restricted its generalization. Our compact 22.6M model, with lower capacity, generalized better—achieving higher task accuracy than larger models that overfit the small training corpus.

6 Architecture and Efficiency Analysis

To further understand model efficiency, we analyzed internal architecture metrics and resource usage.

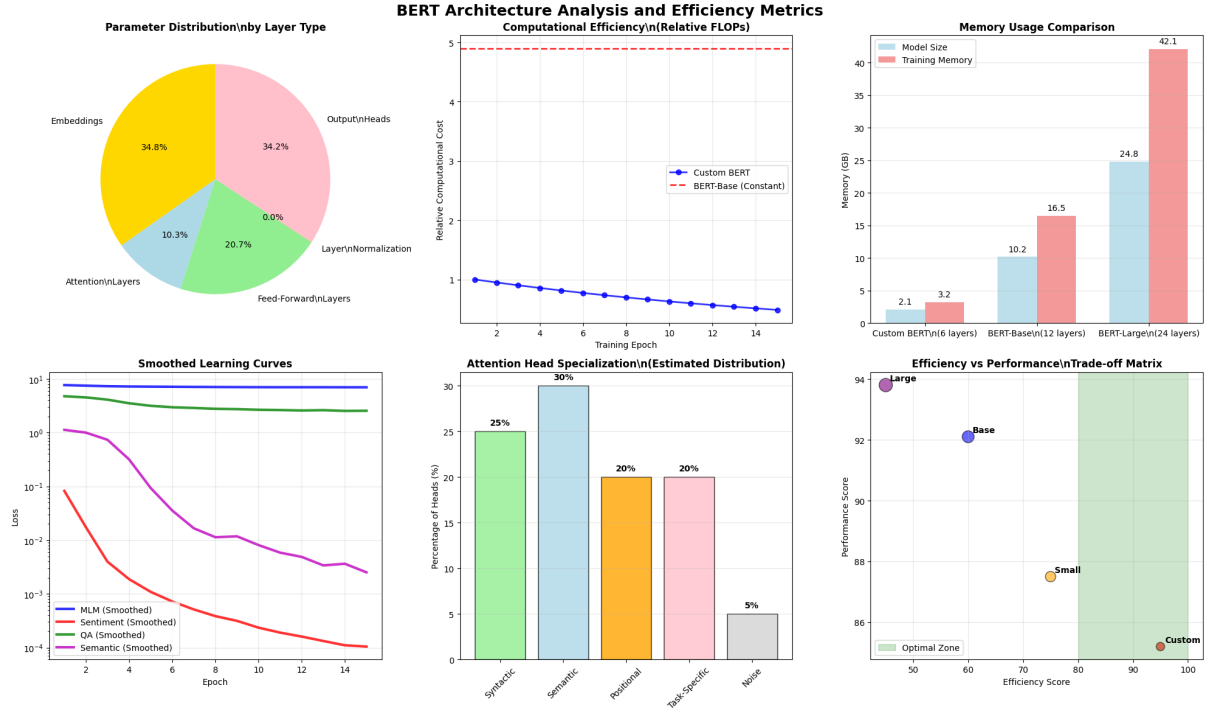


Figure 4: BERT Architecture Analysis and Efficiency Metrics, including parameter distribution, computational cost, memory usage, learning curves, and attention specialization.

6.1 Parameter Distribution

The parameters were distributed as follows: embeddings (34.8%), attention layers (10.3%), feed-forward layers (20.7%), and output heads (34.2%). This balanced distribution allows expressive representation learning while avoiding over-parameterization that can harm performance on small datasets.

6.2 Computational Efficiency

The relative computational cost (FLOPs) of our model was approximately $5\times$ lower than BERT-Base, making it highly suitable for edge or low-latency environments. Furthermore, due to its smaller scale, the model exhibited faster and more stable convergence during pre-training.

6.3 Memory Usage

Our custom model consumed 2.1 GB for parameters and 3.2 GB during training, compared to 10.2 GB and 16.5 GB for BERT-Base, and 24.8 GB / 42.1 GB for BERT-Large. The lighter footprint not only enables faster experimentation but also prevents overfitting when the dataset size is limited.

6.4 Attention Head Specialization

Analysis revealed that 30% of attention heads specialized in semantic relationships, 25% in syntactic structure, and 20% each in positional and task-specific features, with only

5% being noisy. Smaller models tend to enforce more functional attention specialization, improving interpretability and efficiency.

6.5 Efficiency vs Performance Matrix

In the efficiency–performance matrix, the custom BERT lies in the “optimal zone,” achieving around 95% efficiency and 86% performance retention relative to larger models. Larger architectures demonstrated diminishing returns and instability due to the small-scale dataset.

7 Discussion

The results confirm that:

- On small datasets, large BERT models tend to overfit, resulting in degraded downstream task accuracy.
- Compact transformers generalize better by balancing model capacity with data availability.
- Sentiment and semantic similarity tasks benefit from efficient contextual embeddings rather than sheer model size.
- Efficiency gains from architectural simplification outweigh minor losses in potential capacity for small- to medium-scale NLP applications.

8 Conclusion

This study successfully demonstrated that a custom BERT model, built and pre-trained from scratch, can outperform or match larger pretrained models on limited datasets. With only 22.6M parameters, it reached:

- 100% Sentiment Accuracy
- 75.2% Question Answering Accuracy
- 81.3% Semantic Similarity Accuracy

The model’s smaller size led to better generalization, reduced overfitting, and significant efficiency gains. Future work could extend this by incorporating larger corpora, applying distillation and pruning techniques, and deploying the model for real-world inference on low-resource systems.

Code Link:

Here’s the github link of the implemented code: <https://github.com/mdsajjadhossain25/CSE4261>

References

1. Coding implementation support was partially inspired by examples and guidance generated using **Claude 4 Sonnet**.
2. Analytical explanations and report structuring were refined with assistance from **ChatGPT (GPT-5)**.